

Lista 03 - Victor Yaegashi Setti - RA 206362

Questão 1

- Assumindo que o vetor `V` é sempre 1-indexado e possui tamanho `n`
- Assumindo que a função `Swap(V, i, j)` realiza a troca dos valores de `V[i]` com `V[j]`

```
MinHeapify(V, n, i):  
01 esquerdo = 2 * i  
02 direito = 2 * i + 1  
03 menor = i  
04  
05 if (esquerdo <= n && V[esquerdo] < V[menor]):  
06     menor = esquerdo  
07  
08 if (direito <= n && V[direito] < V[menor]):  
09     menor = direito  
10  
11 if (menor != i):  
12     Swap(V, i, menor)  
13     MinHeapify(V, n, menor)
```

```
BuildMinHeap(V, n):  
01 for (i = floor(n / 2); i >= 1; i--):  
02     MinHeapify(V, i, n)
```

```
InvertHeapsort(V, n):  
01  BuildMinHeap(V, n)  
02  
03  for (i = n; i > 1; i--):  
04      Swap(V, 1, i)  
05      n -= 1  
06      MinHeapify(V, 1, n)
```

Questão 2

- Sabendo que o heap possuirá uma tupla `(valor, index)` ao invés de simplesmente o campo `valor`
- Utilizando funções semelhantes às já ensinadas em aula
- Assumindo que os vetores são 1-indexados

```

    KMerge(k, n, vetores):
01  index_atual = CreateVector(k)
02  heap = CreateEmptyMinHeap(k)
03  tamanho_heap = k
04
05  for (i = 1; i <= k; i++):
06      heap[i] = (vetores[i][1], i)
07
08  BuildMinHeap(heap, k)
09
10  V = CreateVector(n)
11  for (i = 1; i <= n; i++):
12      (min, v_origem) = heap[1]
13      V[i] = min
14      index_atual[v_origem] += 1
15
16      if (index_atual[v_origem] > n/k ):
17          Swap(heap, tamanho_heap, 1)
18          tamanho_heap -= 1
19      else:
20          proxima_pos = index_atual[v_origem]
21          heap[1] = (vetores[v_origem][proxima_pos], v_origem)
22      MinHeapify(heap, 1, tamanho_heap)
23  return V

```

Questão 3

Demonstrando por Contradição:

Suponha, por absurdo, que exista um algoritmo de ordenação por comparação com complexidade temporal $O(n)$ para pelo menos metade das permutações de n entradas.

⇒ Pelo menos $\frac{n!}{2}$ entradas levam à folhas com profundidade menor ou igual a $c \cdot n$, sendo $c \in \mathbb{R}_{>0}$ e constante.

Sabe-se que em uma árvore binária, como a de decisão, com altura k pode ter, no máximo, 2^k folhas.

⇒ O número máximo de folhas que podem existir até a profundidade $c \cdot n$ é $2^{c \cdot n}$.

Como cada uma das $\frac{n!}{2}$ entradas deve levar à uma folha diferente temos que:

$$\frac{n!}{2} \leq 2^{c \cdot n}$$

Aplicando $\log_2$ dos dois lados da inequação:

$$\log_2\left(\frac{n!}{2}\right) \leq \log_2(2^{c \cdot n})$$

Aplicando algumas propriedades do $\log_2$:

$$\log_2(n!) - 1 \leq c \cdot n$$

Aplicando a aproximação de Stirling: $\log_2(n!) \geq \frac{n}{2} \log_2\left(\frac{n}{2}\right)$:

$$\frac{n}{2} \log_2\left(\frac{n}{2}\right) - 1 \leq c \cdot n$$

$$\frac{n}{2} (\log_2(n) - 1) - 1 \leq c \cdot n$$

Dividindo ambos os lados da inequação por n :

$$\frac{1}{2} \log_2(n) - \frac{1}{2} - \frac{1}{n} \leq c$$

Como $\log_2(n)$ cresce com relação a n , teríamos que a constante c dependeria de n , o que gera uma contradição com a hipótese inicial.

Questão 4

- Assumindo que a altura de uma árvore é 1-indexado

```

CalculaAlturaEDiametro(A):
01  if (A == NULL):
02      return (0, 0)
03
04  (alt_esq, diam_esq) = CalculaAlturaEDiametro(A.esq)
05  (alt_dir, diam_dir) = CalculaAlturaEDiametro(A.dir)
06
07  alt_atual = max(alt_esq, alt_dir) + 1
08  diam_por_mim = alt_esq + alt_dir
09
10  diam_max_atual = max(diam_por_mim, diam_esq, diam_dir)
11
12  return (alt_atual, diam_max_atual)

```

```
EncontraDiametroTotal(A):  
01  (altura, diametro) = CalculaAlturaEDiametro(A)  
02  return diametro
```

Questão 5

a) Algoritmo de complexidade $O(n \log(k))$

- Sabendo que o heap possuirá uma tupla `(nota, index)` ao invés de simplesmente o campo `nota`
- Utilizando funções semelhantes às já ensinadas em aula
- Assumindo que o vetor `v` é 1-indexado

```
TopCandidatos(V, n, k):  
01  heap = CreateEmptyMinHeap()  
02  
03  for (i = 1; i <= k; i++):  
04      Insert(heap, (V[i], i))  
05  
06  for (i = k+1; i <= n; i++):  
07      (nota_raiz, index_raiz) = GetMin(heap)  
08  
09      if (V[i] > nota_raiz):  
10          RemoveMin(heap)  
11          Insert(heap, (V[i], i))  
12  
13  resultados = CreateVector(k)  
14  for (i = 1; i <= k; i++):  
15      (nota, index) = GetMin(heap)  
16      RemoveMin(heap)  
17      resultados[i] = index  
18  
19  return resultados
```

b) Algoritmo de complexidade $O(n + l)$

- Assumindo que o vetor `v` é 0-indexado

```
TopCandidatosCountingSort(V, n, k, l):  
01 contador = CreateVector(l + 1)  
02  
03 for (i = 0; i < n; i++):  
04     nota = V[i]  
05     contador[nota] += 1  
06  
07 posicao_atual = 0  
08 for (i = l; i >= 0; i--):  
09     frequencia = contador[i]  
10     contador[i] = posicao_atual  
11     posicao_atual += frequencia  
12  
13 temp = CreateVector(n)  
14 for (i = 0; i < n; i++):  
15     nota = V[i]  
16     posicao_destino = contador[nota]  
17     temp[posicao_destino] = i  
18     contador[nota] += 1  
19  
20 resultados = CreateVector(k)  
21 for (i = 0; i < k; i++):  
22     resultados[i] = temp[i]  
23  
24 return resultados
```

Questão 6

- Modificando a função `Mediana(V)` para `Mediana(V, i, f)` por conveniência
- Assumindo que a função `Mediana(V, i, f)` não modifica o vetor `v`

```
    QuicksortMediana(V, i, f):
01  if (i >= f):
02      return
03
04  valor_pivo = Mediana(V, i, f)
05
06  index_pivo = Particiona(V, i, f, valor_pivo)
07
08  QuicksortMediana(V, i, index_pivo - 1)
09  QuicksortMediana(V, index_pivo + 1, f)
```

Questão 7

```
    EncontraTamanhoVetor(A):
01  if (A[0] == false):
02      return 0
03
04  i = 1
05  while (true):
06      if (A[i] == false):
07          break
08      i *= 2
09
10  esquerda = i / 2
11  direita = i
12  n = 0
13
14  while (esquerda <= direita):
15      meio = floor((esquerda + direita) / 2)
16      if (A[meio] == false):
17          direita = meio - 1
18      else:
19          n = meio + 1
20          esquerda = meio + 1
21  return n
```

```

BuscaBinariaSemTamanho(A, x):
01  n = EncontrarTamanhoVetor(A)
02
03  if (n == 0):
04      return -1
05
06  esquerda = 0
07  direita = n - 1
08
09  while (esquerda <= direita):
10      meio = floor((esquerda + direita) / 2)
11      if (A[meio] == x):
12          return meio
13      else if (A[meio] < x):
14          esquerda = meio + 1
15      else:
16          direita = meio - 1
17  return -1

```

Como o algoritmo `EncontrarTamanhoVetor(A)` possui complexidade $O(\log(n))$, temos que o algoritmo `BuscaBinariaSemTamanho(A, x)` possui também complexidade $O(\log(n))$.